EECS 348 – Assignment 3

GenAI Analysis

1. GenAI Programs and Access

The first GenAI program used was ChatGPT by OpenAI. I accessed ChatGPT through the ChatGPT website in a web browser.

The second GenAI program used was Google Gemini. I accessed Gemini through the official Google Gemini website in a web browser.

2. Prompt Used

The exact same prompt was given to both GenAI programs:

Give me the complete C++ code for the following program requirements.

The program will prioritize emails for a busy company CEO.

Use object-oriented programming. The solution must use objects and classes, not a function-oriented design.

Use a MaxHeap as a means of implementing a priority queue. The MaxHeap MUST use a list-based implementation. Implement the MaxHeap and all required behavior from scratch. Do not use any pre-existing heap modules or heap libraries.

Create appropriate C++ classes/objects to represent the emails and the MaxHeap.

The program must process commands in this format:

EMAIL <sender category>,<subject line>,<date>

Sender categories have these priorities, from highest to lowest:

1. Boss
2. Subordinate
3. Peer
4. ImportantPerson
5. OtherPerson

If multiple emails have the same sender-category priority, the newest email must be read first.

Assume:

- sender category is one of the five strings above
- subject line may contain spaces but not commas
- date is in MM-DD-YYYY format

The program must also support these commands:

NEXT
Display the next email for the CEO to read without removing it. Output exactly:

Next email:
Sender: <sender category>
Subject: <subject line>
Date: <date>

READ
Remove the highest-priority unread email from the queue. READ itself should not display the email.

COUNT
Display the current number of unread emails exactly in this format:
There are <count> emails to read.

The program must correctly handle:

- NEXT when there are no emails

- READ when there are no emails

- two NEXT commands in a row without a READ; both NEXT outputs must show the same email

- two READ commands in a row without a NEXT; each READ removes the current highest-priority email without displaying it

- any order and number of commands

- a different grader test file

Sample input:

```
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT
```

Required sample output:

```
There are 6 emails to read.
Next email:
Sender: Boss
Subject: Never Mind
Date: 01-03-2025
Next email:
Sender: Boss
Subject: Important
Date: 12-20-2024
There are 4 emails to read.
```

Make the program usable with redirected standard input from a test file. It may also optionally accept a test filename as a command-line argument.

Return only the complete raw C++ source code.

---

3. Raw Code Generated by ChatGPT

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>

class Email {
private:
    std::string sender;
    std::string subject;
    std::string date;
    int categoryPriority;
    int dateValue;
    long arrivalId;

    int determineCategoryPriority(const std::string& category) const {
        if (category == "Boss") return 5;
        if (category == "Subordinate") return 4;
        if (category == "Peer") return 3;
        if (category == "ImportantPerson") return 2;
        if (category == "OtherPerson") return 1;
        return 0;
    }

    int convertDate(const std::string& dateString) const {
        int month;
        int day;
        int year;
        char dash1;
        char dash2;

        std::stringstream stream(dateString);
        stream >> month >> dash1 >> day >> dash2 >> year;

        return year * 10000 + month * 100 + day;
    }

public:
    Email(const std::string& senderValue,
          const std::string& subjectValue,
          const std::string& dateValueString,
```

```
          long arrivalValue)
        : sender(senderValue),
          subject(subjectValue),
          date(dateValueString),
          arrivalId(arrivalValue) {
        categoryPriority = determineCategoryPriority(sender);
        dateValue = convertDate(date);
    }

    bool hasHigherPriorityThan(const Email& other) const {
        if (categoryPriority != other.categoryPriority) {
            return categoryPriority > other.categoryPriority;
        }

        if (dateValue != other.dateValue) {
            return dateValue > other.dateValue;
        }

        return arrivalId < other.arrivalId;
    }

    const std::string& getSender() const {
        return sender;
    }

    const std::string& getSubject() const {
        return subject;
    }

    const std::string& getDate() const {
        return date;
    }
};

class MaxHeap {
private:
    std::vector<Email> heap;

    void swapEmails(int first, int second) {
        Email temp = heap[first];
        heap[first] = heap[second];
        heap[second] = temp;
    }

    void heapifyUp(int index) {
        while (index > 0) {
            int parent = (index - 1) / 2;

            if (!heap[index].hasHigherPriorityThan(heap[parent])) {
                break;
            }

            swapEmails(index, parent);
            index = parent;
        }
    }
```

```
    void heapifyDown(int index) {
        while (true) {
            int left = 2 * index + 1;
            int right = 2 * index + 2;
            int largest = index;

            if (left < static_cast<int>(heap.size()) &&
                heap[left].hasHigherPriorityThan(heap[largest])) {
                largest = left;
            }

            if (right < static_cast<int>(heap.size()) &&
                heap[right].hasHigherPriorityThan(heap[largest])) {
                largest = right;
            }

            if (largest == index) {
                break;
            }

            swapEmails(index, largest);
            index = largest;
        }
    }

public:
    void insert(const Email& email) {
        heap.push_back(email);
        heapifyUp(static_cast<int>(heap.size()) - 1);
    }

    const Email* peek() const {
        if (heap.empty()) {
            return nullptr;
        }

        return &heap[0];
    }

    void removeMax() {
        if (heap.empty()) {
            return;
        }

        heap[0] = heap.back();
        heap.pop_back();

        if (!heap.empty()) {
            heapifyDown(0);
        }
    }

    int count() const {
        return static_cast<int>(heap.size());
    }
```

```
};

class EmailPriorityProgram {
private:
    MaxHeap inbox;
    long arrivalCounter;

    bool parseEmail(const std::string& line,
                    std::string& sender,
                    std::string& subject,
                    std::string& date) const {
        std::string payload = line.substr(6);

        size_t firstComma = payload.find(',');
        if (firstComma == std::string::npos) {
            return false;
        }

        size_t secondComma = payload.find(',', firstComma + 1);
        if (secondComma == std::string::npos) {
            return false;
        }

        sender = payload.substr(0, firstComma);
        subject = payload.substr(firstComma + 1,
                                 secondComma - firstComma - 1);
        date = payload.substr(secondComma + 1);

        return true;
    }

    void processEmailCommand(const std::string& line) {
        std::string sender;
        std::string subject;
        std::string date;

        if (parseEmail(line, sender, subject, date)) {
            Email email(sender, subject, date, arrivalCounter);
            arrivalCounter++;
            inbox.insert(email);
        }
    }

    void processNextCommand() const {
        const Email* nextEmail = inbox.peek();

        if (nextEmail != nullptr) {
            std::cout << "Next email:" << std::endl;
            std::cout << "Sender: " << nextEmail->getSender() << std::endl;
            std::cout << "Subject: " << nextEmail->getSubject() << std::endl;
            std::cout << "Date: " << nextEmail->getDate() << std::endl;
        }
    }

    void processReadCommand() {
        inbox.removeMax();
```

```
    }

    void processCountCommand() const {
        std::cout << "There are " << inbox.count()
                  << " emails to read." << std::endl;
    }

public:
    EmailPriorityProgram() : arrivalCounter(0) {
    }

    void run(std::istream& input) {
        std::string line;

        while (std::getline(input, line)) {
            if (line.rfind("EMAIL ", 0) == 0) {
                processEmailCommand(line);
            } else if (line == "NEXT") {
                processNextCommand();
            } else if (line == "READ") {
                processReadCommand();
            } else if (line == "COUNT") {
                processCountCommand();
            }
        }
    }
};

int main(int argc, char* argv[]) {
    EmailPriorityProgram program;

    if (argc > 1) {
        std::ifstream inputFile(argv[1]);

        if (!inputFile.is_open()) {
            return 1;
        }

        program.run(inputFile);
    } else {
        program.run(std::cin);
    }

    return 0;
}
```

---

4. Raw Code Generated by Google Gemini

```
#include <iostream>
#include <string>
#include <sstream>
#include <fstream>
#include <vector>
#include <algorithm>

// Enumeration for Sender Categories to easily compare priority
enum class SenderCategory {
    Boss = 1,
    Subordinate = 2,
    Peer = 3,
    ImportantPerson = 4,
    OtherPerson = 5
};

// Helper function to map category string to enum
SenderCategory parseCategory(const std::string& catStr) {
    if (catStr == "Boss") return SenderCategory::Boss;
    if (catStr == "Subordinate") return SenderCategory::Subordinate;
    if (catStr == "Peer") return SenderCategory::Peer;
    if (catStr == "ImportantPerson") return SenderCategory::ImportantPerson;
    return SenderCategory::OtherPerson;
}

// Class to represent an Email
class Email {
private:
    std::string senderCategoryStr;
    SenderCategory category;
    std::string subject;
    std::string dateStr;
    std::string normalizedDate; // "YYYY-MM-DD" for chronological sorting

    std::string convertToNormalizedDate(const std::string& date) const {
        if (date.length() == 10 && date[2] == '-' && date[5] == '-') {
            std::string mm = date.substr(0, 2);
            std::string dd = date.substr(3, 2);
            std::string yyyy = date.substr(6, 4);
            return yyyy + "-" + mm + "-" + dd;
        }
        return date;
    }

public:
    Email(const std::string& categoryStr, const std::string& subjectLine, const std::string& date)
        : senderCategoryStr(categoryStr),
          category(parseCategory(categoryStr)),
          subject(subjectLine),
          dateStr(date) {
        normalizedDate = convertToNormalizedDate(date);
    }

    const std::string& getSenderCategory() const { return senderCategoryStr; }
    const std::string& getSubject() const { return subject; }
    const std::string& getDate() const { return dateStr; }

    // Operator to define priority comparison for MaxHeap.
    // Higher priority email returns true when evaluated as greater than another.
    bool operator>(const Email& other) const {
        // Priority 1: Lower numeric value of SenderCategory means higher priority
        if (this->category != other.category) {
```

```
            return static_cast<int>(this->category) < static_cast<int>(other.category);
        }
        // Priority 2: If sender priority is the same, newer date is prioritized (lexicographically greater YYYY-MM-DD)
        return this->normalizedDate > other.normalizedDate;
    }

    bool operator<(const Email& other) const {
        if (this->category != other.category) {
            return static_cast<int>(this->category) > static_cast<int>(other.category);
        }
        return this->normalizedDate < other.normalizedDate;
    }
};

// Node structure for List-based MaxHeap
template <typename T>
struct Node {
    T data;
    Node* left;
    Node* right;
    Node* parent;

    Node(T val) : data(val), left(nullptr), right(nullptr), parent(nullptr) {}
};

// Custom MaxHeap class implemented using a dynamic tree/list node structure
template <typename T>
class MaxHeap {
private:
    Node<T>* root;
    size_t heapSize;

    // Returns a pointer to the node at the 1-based index using binary bit manipulation
    Node<T>* getNodeAt(size_t index) const {
        if (index == 0 || index > heapSize) return nullptr;
        if (index == 1) return root;

        // Find binary path from root
        std::vector<int> path;
        while (index > 1) {
            path.push_back(index % 2); // 0 for left child, 1 for right child
            index /= 2;
        }

        Node<T>* current = root;
        for (int i = static_cast<int>(path.size()) - 1; i >= 0; --i) {
            if (path[i] == 0) {
                current = current->left;
            } else {
                current = current->right;
            }
        }
        return current;
    }

    void heapifyUp(Node<T>* node) {
        while (node->parent != nullptr && node->data > node->parent->data) {
            std::swap(node->data, node->parent->data);
            node = node->parent;
        }
    }
```

```
    void heapifyDown(Node<T>* node) {
        while (node != nullptr) {
            Node<T>* largest = node;

            if (node->left != nullptr && node->left->data > largest->data) {
                largest = node->left;
            }
            if (node->right != nullptr && node->right->data > largest->data) {
                largest = node->right;
            }

            if (largest != node) {
                std::swap(node->data, largest->data);
                node = largest;
            } else {
                break;
            }
        }
    }

    void clearTree(Node<T>* node) {
        if (node == nullptr) return;
        clearTree(node->left);
        clearTree(node->right);
        delete node;
    }

public:
    MaxHeap() : root(nullptr), heapSize(0) {}

    ~MaxHeap() {
        clearTree(root);
    }

    size_t size() const {
        return heapSize;
    }

    bool empty() const {
        return heapSize == 0;
    }

    void push(const T& val) {
        heapSize++;
        Node<T>* newNode = new Node<T>(val);

        if (heapSize == 1) {
            root = newNode;
            return;
        }

        // Insert at position `heapSize`
        Node<T>* parentNode = getNodeAt(heapSize / 2);
        newNode->parent = parentNode;

        if (heapSize % 2 == 0) {
            parentNode->left = newNode;
        } else {
            parentNode->right = newNode;
        }

        heapifyUp(newNode);
```

```
    }

    const T& top() const {
        return root->data;
    }

    void pop() {
        if (empty()) return;

        if (heapSize == 1) {
            delete root;
            root = nullptr;
            heapSize = 0;
            return;
        }

        Node<T>* lastNode = getNodeAt(heapSize);
        root->data = lastNode->data;

        // Detach lastNode from parent
        Node<T>* parentNode = lastNode->parent;
        if (parentNode->left == lastNode) {
            parentNode->left = nullptr;
        } else {
            parentNode->right = nullptr;
        }

        delete lastNode;
        heapSize--;

        heapifyDown(root);
    }
};

// Priority Queue for Managing Emails
class EmailPriorityQueue {
private:
    MaxHeap<Email> heap;

public:
    void addEmail(const Email& email) {
        heap.push(email);
    }

    void displayNext() const {
        if (heap.empty()) {
            return;
        }
        const Email& nextEmail = heap.top();
        std::cout << "Next email:\n";
        std::cout << "Sender: " << nextEmail.getSenderCategory() << "\n";
        std::cout << "Subject: " << nextEmail.getSubject() << "\n";
        std::cout << "Date: " << nextEmail.getDate() << "\n";
    }

    void readNext() {
        if (!heap.empty()) {
            heap.pop();
        }
    }

    void displayCount() const {
```

```
        std::cout << "There are " << heap.size() << " emails to read.\n";
    }
};

void processStream(std::istream& in) {
    EmailPriorityQueue emailQueue;
    std::string line;

    while (std::getline(in, line)) {
        // Strip carriage returns if present
        if (!line.empty() && line.back() == '\r') {
            line.pop_back();
        }

        if (line.empty()) continue;

        if (line.rfind("EMAIL ", 0) == 0) {
            std::string emailDetails = line.substr(6);
            std::stringstream ss(emailDetails);
            std::string category, subject, date;

            if (std::getline(ss, category, ',') &&
                std::getline(ss, subject, ',') &&
                std::getline(ss, date, ',')) {
                Email email(category, subject, date);
                emailQueue.addEmail(email);
            }
        } else if (line == "NEXT") {
            emailQueue.displayNext();
        } else if (line == "READ") {
            emailQueue.readNext();
        } else if (line == "COUNT") {
            emailQueue.displayCount();
        }
    }
}

int main(int argc, char* argv[]) {
    if (argc > 1) {
        std::ifstream file(argv[1]);
        if (file.is_open()) {
            processStream(file);
            file.close();
        } else {
            std::cerr << "Error: Could not open file " << argv[1] << std::endl;
            return 1;
        }
    } else {
        processStream(std::cin);
    }

    return 0;
}
```

5. Code Analysis

Correctness

I found that both programs handle the required EMAIL, NEXT, READ, and COUNT commands. Both also prioritize emails by sender category first and then by date, so newer emails from the same category are read first but I think the ChatGPT version matches the assignment better because it uses a vector for the MaxHeap and organizes the program using Email, MaxHeap, and EmailPriorityProgram classes.

The Gemini version also uses classes, but its MaxHeap is built using linked Node objects with left, right, and parent pointers. Since the assignment specifically asks for a list-based MaxHeap, I think this could be a problem. Gemini also uses some standalone functions outside the classes, which makes it feel less object-oriented than the ChatGPT version.

Both programs safely handle NEXT and READ when there are no emails. They also handle repeated NEXT commands without removing the email and repeated READ commands without displaying the removed email.

Execution Time

For the ChatGPT version, adding an email takes O(log n) because the email may need to move up the heap. READ also takes O(log n) because the heap may need to be reorganized after removing the highest-priority email.

NEXT is O(1) because it only looks at the first email in the heap. COUNT is also O(1) because it only checks the current heap size.

The Gemini version also has O(log n) insertion and removal, but it has to move through linked nodes to find positions in the heap. This adds extra work compared to directly accessing elements in a vector.

Space Complexity

Both programs use O(n) space because they have to store all unread emails.

The ChatGPT version stores the emails directly in a vector.

The Gemini version stores each email inside a Node that also contains left, right, and parent pointers. Because of this, the Gemini version uses more memory for each email.

Maintainability

I found the ChatGPT version easier to understand and maintain. It separates the program into clear classes: Email stores the email information, MaxHeap manages the priority queue, and EmailPriorityProgram handles the commands.

The Gemini version is more complicated because it uses templates, linked nodes, multiple pointers, operator overloading, and manual memory cleanup. This gives the program more places where mistakes could happen.

Overall, I think the ChatGPT version is simpler, easier to follow, uses less extra memory, and matches the object-oriented and list-based requirements better.

---

6. Program Selection and Justification

I selected the ChatGPT program as the basis for my final program.
Both programs correctly handle the main email-priority behavior, but I chose ChatGPT because its design matches the assignment requirements more closely.

The ChatGPT version uses a vector-based MaxHeap, which better fits the required list-based implementation. It also organizes most of the behavior into the Email, MaxHeap, and EmailPriorityProgram classes, so the design feels more object-oriented.

I also found the ChatGPT version simpler to follow. The Gemini version uses linked nodes, several pointers, templates, and manual memory cleanup, which makes it more complicated and uses more memory for each email.

Because the ChatGPT version is simpler, easier to maintain, and closer to the required design, I think it is the better starting point for the final program.

---

7. Improvements Made

Correctness

I kept the ChatGPT version's vector-based MaxHeap because it matches the list-based requirement better than Gemini's linked-node version. I also kept the empty NEXT and READ behavior so the program safely does nothing when there are no emails instead of printing extra output that the grader may not expect.

I also kept the tie-breaking rule for emails with the same sender category and date so the ordering is consistent.

Execution Time

I kept the MaxHeap structure because EMAIL and READ already run in O(log n) time, while NEXT and COUNT run in O(1) time. I also kept the vector-based design because it allows direct access to heap positions without having to move through linked nodes.

Space Complexity

I kept the vector-based MaxHeap because it stores the Email objects directly instead of creating a separate node with multiple pointers for every email.

The program still uses O(n) space because it must store all unread emails, but it avoids the extra pointer storage used in the Gemini version.

Maintainability

I kept the program organized around the Email, MaxHeap, and EmailPriorityProgram classes.

I added a complete prologue, clearer comments, and source attribution. I also made the class responsibilities clear so it is easier to understand which part of the program stores email data, manages the heap, and handles commands.

These changes make the final program easier to read, test, and modify while keeping the object-oriented design required by the assignment.
